Navigation

- index
- modules |
- next |
- previous |
- PIPESIM Python Toolkit 2023.1 documentation »
- Quick Start

Quick Start¶

The following section provides a quick reference guide to the properties, methods and classes available in the PIPESIM Python Toolkit. Working example scripts can be found in the PIPESIM installation under the Python Toolkit folder.

The easiest way to investigate a model is through an interactive Python session. Code is executed as it is entered and the results are immediately seen.

From the Windows Start menu, open and start VSCodium. Type the following commands into the interactive window to see the effect of performing them.

Open, Save and Close¶

Open a model:

```
>>> from sixgill.pipesim import Model
>>> model = Model.open('C:/Temp/examples/models/CSN_301_Small Network.pips')
```

Open a model and use the SI units of measurement for passing data:

```
>>> model = Model.open(
    filename='C:/Temp/examples/models/CSN_301_Small Network.pips',
    units=Units.SI)
```

Query details about the model:

```
>>> model.about()
```

Close the model:

```
>>> model.close()
```

Save the model:

```
>>> model.save()
```

Save the model as a new filename:

```
>>> model.save('C:/Temp/myproject/Modified Small Network.pips')
```

Save the model using Network perspective:

```
>>> from sixgill.core.resources import ViewTypes
>>> model.save('C:/Temp/myproject/Modified Small Network.pips',ViewTypes.NETWORK)
```

Create a new model:

```
>>> model = Model.new('C:/Temp/myproject/New Small Network.pips')
```

Open the current model in the UI:

This will close the model in the Python Toolkit as the model cannot be edited at the same time in the Toolkit and UI.

```
>>> model.open_ui()
```

Open a specific model in the UI:

```
>>> Model.open_ui('C:/Temp/myproject/New Small Network.pips')
```

Query the Model¶

Operations on the model start by providing the context for the operation. The context defines what model components are then acted on. The simplest method is find() which returns a list of the model components within the context.

List out all the components in the model:

```
>>> model.find(Name=ALL)
```

Look for all the chokes in the model:

```
>>> from sixgill.definitions import *
>>> model.find(component=ModelComponents.CHOKE)
```

Alternatively, if we already know the choke keyword:

```
>>> model.find(Choke=ALL)
```

Find if an equipment exists by looking up its name:

```
>>> model.find(Name='PMP')
```

Find the tubing in Well_1:

```
>>> model.find(component=ModelComponents.TUBING, Well='Well_1')
```

The tubing can also be referenced by its context:

```
>>> model.find(context="Well_1:Tubing_1")
```

Find the context for all the completions on all the wells:

```
>>> model.find(Well=ALL, component=ModelComponents.COMPLETION)
```

Large models can be reduced into smaller models through a filtered model which contains just those items in the context. The filtered model has all the same methods available as the full model. For example, selecting a well to perform actions on:

```
>>> wellmodel = model.filter(Well="Well_1")
>>> wellmodel.find(component=ModelComponents.COMPLETION)
```

Read and Write Parameters¶

The parameters for the model components are accessed through the get_value() and set_value() methods to read and write values respectively. There are also get_values() and set_values() methods to read and write many parameter components at the same time.

Get the bean size of a choke:

```
>>> from sixgill.definitions import *
>>> beansize = model.get_value(Name = 'Choke', parameter = Parameters.Choke.BEANSIZE)
```

In this case the context for the choke is the same as its name, so alternatively:

```
>>> model.get_value('Choke', Parameters.Choke.BEANSIZE)
```

The Parameters static class defines the parameters as strings. The following style can be used but it is not recommended:

```
>>> model.get_value('Well_1:VertComp', 'GeometryProfileType')
```

Set the new bean size for the choke:

```
>>> model.set_value(Name = 'Choke', parameter = Parameters.Choke.BEANSIZE,value = 12)
```

Or simply, by using the context and argument ordering:

```
>>> model.set_value('Choke', Parameters.Choke.BEANSIZE, 12)
```

Parameters can be referenced from the top level context, so long as they are unique. To set a new reservoir pressure on Well_1 with only one completion:

```
>>> model.set_value(
    Well='Well_1',
    parameter=Parameters.Completion.RESERVOIRPRESSURE,
    value='3210')
```

Which is the equivalent to:

```
>>> model.set_value('Well_1:VertComp', Parameters.Completion.RESERVOIRPRESSURE, '3210')
```

To retrieve multiple parameters from multiple components, the get_values() method is used. For example to get specific parameters for the choke:

```
>>> model.get_values(
    Name='Choke',
    parameters=[Parameters.Choke.BEANSIZE,
    Parameters.Choke.CRITICALPRESSURERATIO])
```

This returns a dictionary of the components, each having a dictionary of the parameters. To retrieve the parameters from all the chokes:

```
>>> model.get_values(
    Choke=ALL,
    parameters=[Parameters.Choke.BEANSIZE,
    Parameters.Choke.CRITICALPRESSURERATIO])
```

Passing in an empty parameter list or not passing in a list returns all the parameters for the context. To get all the parameters for all the compressors in the model:

```
>>> params = model.get_values(Compressor=ALL)
```

To update the compressors with new parameters, pass in a dictionary of the compressor parameters:

```
>>> updated_params = {'CentComp': {
    Parameters.Compressor.HONOURSTONEWALLLIMIT: True,
    Parameters.Compressor.ROUTE: CompressorThermodynamics.ADIABATIC,
    Parameters.Compressor.EFFICIENCY: 100.0,
    Parameters.Compressor.ISACTIVE: True,
    Parameters.Compressor.PRESSUREDIFFERENTIAL: 2400.0}}
>>> model.set_values(updated_params)
```

The set_value() method sets the value for a single component and will throw an error if there is more than one component in the context. To set the same parameter value for all the items in the context, use the set_all_value() method:

```
>>> model.set_all_value(
    Choke=ALL,
    parameter=Parameters.Choke.BEANSIZE,
    value=2)
```

Read and Write Data Tables¶

Parameters such as well trajectories, flowline geometries, and geothermal surveys are handled as arrays. The data is passed to and from PIPESIM as pandas DataFrames so that it can be easily manipulated.

To read a flowline geometry using its context:

```
>>> model.get_geometry(context="F_10")
```

Or by passing the name as a context:

```
>>> model.get_geometry(Name="F_10")
```

The well trajectory is handled similarly as:

```
>>> model.get_trajectory(Name="GI_Well")
```

For a geothermal survey:

```
>>> model.get_geothermal_profile(Name="F_10")
```

There are corresponding methods for setting the data tables. The methods take the context and the DataFrame with the tabulated data. The method to set a flowline geometry is:

```
>>> model.set_geometry(context="F_20", dataframe=df)
```

Where df is the pandas DataFrame containing the tabulated flowline profile data. The DataFrame must be in the same format that the corresponding get() function uses, must have the same column headings, and must be indexed from 0.

For getting a pressure/flowrate curve on a source:

```
>>> model.get_pq_curve(Source="SRC1")
```

Add and Delete¶

Add a new source to the model:

```
>>> model.add(component=ModelComponents.SOURCE, name='Src1')
```

The parameters for the model component can be passed as an argument:

```
>>> model.add(
    ModelComponents.CHOKE,
    'CK1', context="Well 2",
    parameters=
    {
        Parameters.Choke.TOPMEASUREDDEPTH:500,
        Parameters.Choke.BEANSIZE:2,
        Parameters.Choke.SUBCRITICALCORRELATION:Constants.SubCriticalFlowCorrelation.ASHFORD
    })
```

For downhole equipment, pass in the context and aspects such as measured depth:

```
>>> model.add(
    component=ModelComponents.ESP,
    name="ESP1",
    context="Well 2",
    parameters=
    {
        Parameters.ESP.TOPMEASUREDDEPTH:1500,
        Parameters.ESP.OPERATINGFREQUENCY:60,
        Parameters.ESP.MANUFACTURER:"ALNAS",
        Parameters.ESP.MODEL:"ANA580",
        Parameters.ESP.NUMBERSTAGES:100,
        Parameters.ESP.HEADFACTOR:1,
        Parameters.ESP.POWERFACTOR:0.95,
        Parameters.ESP.USEVISCOSITYCORRECTION:True
    })
```

Delete a model component:

```
>>> model.delete(Name='Src1')
```

Copy, Convert, and Rename¶

Copy a well and include all the associated equipment:

```
>>> model.copy(context='Well_1', name='Well_2')
```

Convert a junction into a well:

```
>>> model.convert(
    context="Manifold",
    to_component=ModelComponents.WELL)
```

Rename a component by setting the Name parameter:

```
>>> model.set_value(
    context="Src1",
    parameter=Parameters.Source.NAME,
    'SOURCE_1')
```

Connect and Disconnect¶

PIPESIM uses ports to reference the inlet and outlet to components. To connect, specify the source (from) components, the destination (to) components, and the port to be connected where applicable.

To connect a well 'Well_1' to a flowline 'FL-1':

```
>>> model.connect('Well_1', 'FL-1')
```

Separators have two or three outlet ports hence outlet connections from a separator need to explicitly state which is used:

```
>>> model.connect(
    source='MBD101',
    destination='FL-PMP-1',
    source_port=Connection.Separator.TOP)
```

Disconnecting ports is done in the same way:

```
>>> model.disconnect(source='MDB101', destination='FL-PMP-1')
```

The model connections for a context are provided as a dictionary. This can be converted into a pandas DataFrame for convenient reviewing. Omitting the context will list all the connections in the model:

```
>>> model.connections(context='MBD-101')
>>> # Or to see it as a nicely formatted list
>>> import pandas as pd
>>> pd.DataFrame(model.connections(context='MBD-101'))
```

The get_connections() method provides a list of the connections for each item in the context. This method is easier to code in what is connected to a model component

```
>>> mbd101_connections = model.get_connections(context="MBD-101")
>>> mbd101_connections[Connection.SOURCE]
>>> mdb101_connections[Connection.Separator.TOP]
```

Manipulate Fluids¶

Black oil and compositional fluids are treated as any other model component. To select fluid type and compositional fluid details use a fluid method.

Create a black oil fluid:

```
>>> model.add(ModelComponents.BLACKOILFLUID, "BK111")
```

Set the properties of the black oil fluid using, the set_value() method, for example:

```
>>> model.set_value(
    BlackOilFluid="BK111",
    parameter=Parameters.BlackOilFluid.GOR,
    value=200)
>>> model.set_value(
    BlackOilFluid="BK111",
    parameter=Parameters.BlackOilFluid.WATERCUT,
    value=5)
>>> model.set_value(
    BlackOilFluid="BK111",
    parameter=Parameters.BlackOilFluid.GASSPECIFICGRAVITY,
    value=0.7)
>>> model.set_value(
    BlackOilFluid="BK111",
    parameter=Parameters.BlackOilFluid.WATERSPECIFICGRAVITY,
    value=1.05)
>>> model.set_value(
    BlackOilFluid="BK111",
    parameter=Parameters.BlackOilFluid.API,
    value=21)
```

Set calibration data for black oil fluid using, the set_values() method, for example:

```
>>> single_point_calibration = Parameters.BlackOilFluid.SinglePointCalibration
>>> model.set_values(
    {"BK111": {
    single_point_calibration.SOLUTIONGAS:
    Constants.BlackOilCalibrationSolutionGas.VASQUEZANDBEGGS,
    single_point_calibration.BUBBLEPOINTSATGAS_PRESSURE: 2424,
    single_point_calibration.BUBBLEPOINTSATGAS_TEMPERATURE: 250,
    single_point_calibration.BUBBLEPOINTSATGAS_VALUE: 202.63
    }})
```

Set the thermal data for black oil fluid:

```
>>> model.set_value(
    BlackOilFluid="BK111",
    parameter=Parameters.BlackOilFluid.ThermalData.GASHEATCAPACITY,
    value=0.50)
>>> model.set_value(
    BlackOilFluid="BK111",
    parameter=Parameters.BlackOilFluid.ThermalData.OILHEATCAPACITY,
    value=0.40)
>>> model.set_value(
    BlackOilFluid="BK111",
    parameter=Parameters.BlackOilFluid.ThermalData.WATERHEATCAPACITY,
    value=1.01)
>>> model.set_value(
    BlackOilFluid="BK111",
    parameter=Parameters.BlackOilFluid.ThermalData.ENTHALPYCALCMETHOD,
    value=EnthalpyCalcMethod.METHOD2009)
>>> model.set_value(
    BlackOilFluid="BK111",
    parameter=Parameters.BlackOilFluid.ThermalData.LATENTHEATOFVAPORIZATION,
    value=141)
```

Get the parameterization of the black oil fluid:

```
>>> model.get_values(BlackOilFluid="BK111")
```

Select using compositional fluid rather than black oil:

```
>>> model.fluids.fluid_type = FluidType.COMPOSITIONAL
```

Set the compositional fluid options:

```
>>> eos = Constants.EquationOfState.Multiflash.CUBICPLUSASSOCIATION
>>> model.fluids.compositional.pvt_package = Constants.PVTPackage.MULTIFLASH
>>> model.fluids.compositional.equation_of_state = eos
>>> model.fluids.compositional.salinity_model = Constants.SalinityModel.IONANALYSIS
```

Create a custom pseudocomponent:

```
>>> c7plus_component_params = {
    Parameters.FluidComponent.HYDROCARBONFORM:Constants.FluidComponentType.HYDROCARBON,
    Parameters.FluidComponent.MOLECULARWEIGHT:234,
    Parameters.FluidComponent.TBOIL:250,
    }
>>> model.fluids.compositional.add_pseudocomponent(
    "C7Plus",
    parameters=c7plus_component_params)
```

Select compositional fluid components:

```
>>> selected_components = [
    Constants.MultiflashComponent.METHANE,
    Constants.MultiflashComponent.ETHANE,
    Constants.MultiflashComponent.HEXANE,
    Constants.MultiflashComponent.ETHANOL,
    Constants.MultiflashComponent.PROPANE,
    Constants.MultiflashComponent.WATER,
    "C7Plus",
    ]
>>> model.fluids.compositional.select_components(selected_components)
```

Create a compositional fluid with with salt component:

```
>>> fluid_parameters = {
    Parameters.CompositionalFluid.LIQUIDVISCOSITYCALC:
        Constants.EmulsionViscosityMethod.CONTINUOUSPHASE,
    Parameters.CompositionalFluid.USERWATERCUTCUTOFF:40,
    Parameters.CompositionalFluid.IONSODIUM:1000,
    Parameters.CompositionalFluid.IONCALCIUM:5000,
    Parameters.CompositionalFluid.IONMAGNESIUM:3000,
    Parameters.CompositionalFluid.IONPOTASSIUM:4012,
    Parameters.CompositionalFluid.IONSTRONTIUM:5000,
    Parameters.CompositionalFluid.IONBARIUM:6000,
    Parameters.CompositionalFluid.IONIRON:17000,
    Parameters.CompositionalFluid.IONCHLORIDE:1000,
    Parameters.CompositionalFluid.IONSULPHATE:1235,
    Parameters.CompositionalFluid.IONBICARBONATE:11122,
    Parameters.CompositionalFluid.IONBROMIDE:544,
    Parameters.CompositionalFluid.IONBARIUM:5555,
    Parameters.CompositionalFluid.SALTWATERDENSITYTYPE:
        Constants.SaltWaterDensity.DENSITY,
    Parameters.CompositionalFluid.SALTWATERDENSITY:72,
    }
>>> model.add(ModelComponents.COMPOSITIONALFLUID, "Comp1", parameters=fluid_parameters)
```

The compositional fluid is read through the method on the fluids:

```
>>> model.fluids.compositional.get_composition("Comp1")
```

And the corresponding method for setting the composition:

```
>>> model.fluids.compositional.set_composition(df)
```

Assign a black oil fluid to a well:

```
>>> model.set_value(
    Well = "Well 2",
    parameter=Parameters.Well.ASSOCIATEDBLACKOILFLUID,
    value="BK111")
```

Assign a compositional fluid to a well:

```
>>> model.set_value(
    Well = "Well 2",
    parameter=Parameters.Well.ASSOCIATEDCOMPOSITIONALFLUID,
    value="COMP1")
```

Import and Export Wells¶

You can import and export Wells to/from individual PIPESIM models. To export a well to a PIPESIM model:

```
>>> model.export_well(context="Well_1",
    folder="C:/temp/myproject/wells")
```

To export all the wells, leave out the context:

```
>>> model.export_well(folder="C:/temp/myproject/wells")
```

If the folder is not provided then it will export to the same folder as the currently open PIPESIM model.

Import a PIPESIM well model into the currently open model:

```
>>> model.import_well(filename="C:/temp/myproject/Well.pips", name="Well_1")
```

To use the fluid from the imported model:

```
>>> model.import_well(
    filename="C:/temp/myproject/Well.pips",
    name="Well_1",
    fluid_override=False)
```

To overwrite the existing "Well_1" in the current model with the imported well:

```
>>> model.import_well(
    filename="C:/temp/myproject/Well.pips",
    name="Well_1",
    overwrite_existing=True)
```

Catalog and GIS¶

Get the elevation for a coordinate (latitude, longitude):

```
>>> model.get_elevation(lat=44.4532, long=76.16231)
```

Get elevations for multiple coordinates:

```
>>> model.get_elevations(
        [{Parameters.Location.LATITUDE:44.4532,Parameters.Location.LONGITUDE:76.16231},
        {Parameters.Location.LATITUDE:45.5532,Parameters.Location.LONGITUDE: 78.6678}])
```

Reading entries from the catalog. Parameters required for the look-up such as Manufacturer, series, etc. must have been set on the component first:

```
>>> model.read_catalog("PCP")
```

Simulation Settings¶

You can operate on the simulation settings through both a dictionary and individual properties.

Get the current simulation settings as a dictionary:

```
>>> model.sim_settings()
```

Get the ambient temperature:

```
>>> model.sim_settings.ambient_temperature
```

Set the ambient temperature:

```
>>> model.sim_settings.ambient_temperature = 70
```

Alternatively, using the dictionary syntax:

```
>>> model.sim_settings[Parameters.SimulationSetting.AMBIENTTEMPERATURE] = 70
```

The global flow correlations are provided as a dictionary by:

```
>>> model.sim_settings.global_flow_correlation()
```

And to set the global flow correlations, pass back in a dictionary:

```
>>> multiphase = Parameters.FlowCorrelation.Multiphase
>>> model.sim_settings.global_flow_correlation({
    multiphase.Vertical.CORRELATION:
        Constants.MultiphaseFlowCorrelation.BakerJardine.BEGGSBRILL,
    multiphase.Horizontal.CORRELATION:
        Constants.MultiphaseFlowCorrelation.BakerJardine.BEGGSBRILL
    })
```

To use the local flow correlations:

```
>>> model.sim_settings.use_global_flow_correlation = False
```

The local flowline correlations are accessed through context based get/set methods.

```
>>> model.sim_settings.get_flow_correlations(Flowline=ALL)
```

If the context is left empty then it returns all the flowline and well trajectories that be specified:

```
>>> model.sim_settings.get_flow_correlations()
```

To set the flow correlations for a flowline, pass in the dictionary of parameters for the flowlines or wells to be set:

```
>>> multiphase = Parameters.FlowCorrelation.Multiphase
>>> model.sim_settings.set_flow_correlations({
    "FL_10": {
        Parameters.FlowCorrelation.OVERRIDEGLOBAL: True,
        multiphase.Vertical.CORRELATION:
            Constants.MultiphaseFlowCorrelation.BakerJardine.BEGGSBRILL,
        multiphase.Horizontal.CORRELATION:
            Constants.MultiphaseFlowCorrelation.BakerJardine.BEGGSBRILL
    }})
```

The heat transfer options use the same methodology. To get the global heat transfer options:

```
>>> model.sim_settings.global_heat_transfer_option()
```

To set the heat transfer options:

```
>>> model.sim_settings.global_heat_transfer_option({
Parameters.HeatTransferOption.PIPEBURIALMETHOD: Constants.PipeBurialMethod.METHOD2009})
```

To get and set the local heat transfer options:

```
>>> model.sim_settings.get_heat_transfer_options(Flowline=ALL)
>>> model.sim_settings.set_heat_transfer_options({
    "FL_10": {
    Parameters.HeatTransferOption.OVERRIDEGLOBAL: True,
    Parameters.HeatTransferOption.PIPEBURIALMETHOD: Constants.PipeBurialMethod.METHOD2009
    }})
```

The list of available simulation setting properties is provided in the Quick Reference section below.

Simulations¶

Simulations are performed through the tasks class. Each task has the same core methods available, with additional methods depending on the task being performed.

Get the current boundary conditions for a network simulation:

```
>>> model.tasks.networksimulation.get_conditions()
```

For a specific study:

```
>>> model.tasks.networksimulation.get_conditions(study="Late Field Life")
```

Update the boundary conditions for the default network simulation study:

```
>>> boundaries =
    {"Well:VertComp":
        {
            Parameters.Boundary.PRESSURE:NAN,
            Parameters.Boundary.TEMPERATURE:150,
            Parameters.Boundary.FLOWRATETYPE:Constants.FlowRateType.LIQUIDFLOWRATE,
            Parameters.Boundary.LIQUIDFLOWRATE:200
        }
    }
>>> model.tasks.networksimulation.set_conditions(boundaries = boundaries)
```

To reset the boundary conditions to those of the model for the default study:

```
>>> model.tasks.networksimulation.reset_conditions()
```

The simulation performance is directly affected by the number of specified system and profile variables in a simulation task. If omitted, then all the possible system and profile variables are returned, which will significantly jeopardize the simulation performance. To produce all results available in a UI simulation run, the predefined output variables available in the UI can be selected. These predefined output variables include large lists of variables. For the best simulation performance, only the required output variables are specified.

To run the network simulation study:

```
>>> system_variables = [
    SystemVariables.PRESSURE,
    SystemVariables.TEMPERATURE,
    SystemVariables.VOLUME_FLOWRATE_LIQUID_STOCKTANK,
    SystemVariables.VOLUME_FLOWRATE_OIL_STOCKTANK,
    SystemVariables.VOLUME_FLOWRATE_WATER_STOCKTANK,
    SystemVariables.VOLUME_FLOWRATE_GAS_STOCKTANK,
    SystemVariables.BOTTOM_HOLE_PRESSURE,
    SystemVariables.OUTLET_GLR_STOCKTANK,
    SystemVariables.OUTLET_WATER_CUT_STOCKTANK,
    ]
>>> profile_variables = [
    ProfileVariables.TEMPERATURE,
    ProfileVariables.ELEVATION,
    ProfileVariables.TOTAL_DISTANCE,
    ]
>>> results = model.tasks.networksimulation.run(
    system_variables=system_variables,
    profile_variables=profile_variables)
```

The study conditions can be passed into the run_simulation() method. This allows simulations to be queued up, each with their own study conditions:

```
>>> results = model.tasks.networksimulation.run(
    boundaries=boundaries,
    system_variables=system_variables, profile_variables=profile_variables)
```

Predefined results variable lists are available as available in the PIPESIM UI:

```
>>> results = model.tasks.networksimulation.run(
    boundaries=boundaries,
    system_variables=OutputVariables.System.FLOW_ASSURANCE,
    profile_variables=OutputVariables.Profile.FLOW_ASSURANCE)
```

The simulation results is an object with three properties: system, node and profile. For network simulations, the node and system results are dictionaries of each node or system point respectively while the profile is a dictionary of each branch. In a PT Profile simulation the system points is also a dictionary of the sensitivities.

The node results for a network simulation as a dataframe:

```
>>> pd.DataFrame(results.node)
```

The system results for a network simulation as a dataframe

```
>>> pd.DataFrame(results.system)
```

The profile results for flowline FL-45 in a network simulation as a DataFrame:

```
>>> pd.DataFrame(results.profile["FL-45"])
```

The run() method executes concurrently and the results are returned when it finishes. To run the simulation in the background, use the start_simulation() method. This allows the Python script to continue executing while the simulation is running:

```
>>> simulation_id = model.tasks.networksimulation.start(
        profile_variables=profile_variables,
        system_variables=system_variables)
```

In this case, the status of the simulation is returned by:

```
>>> model.tasks.networksimulation.get_state(simulation_id)
```

And when finished, the simulation message and results are retrieved:

```
>>> message = model.tasks.networksimulation.get_messages(simulation_id)
>>> results = model.tasks.networksimulation.get_results(simulation_id)
```

Note that the simulations using start() are queued by the PIPESIM engine and are run concurrently. The simulations are not run in parallel, only that the Python script can continue running while each simulation finishes.

PT Profile simulations are performed in the same manner. The arguments passed to the study and simulation methods are accordingly different:

```
>>> parameters = {
        Parameters.PTProfileSimulation.INLETPRESSURE:2600,  #psia
        Parameters.PTProfileSimulation.OUTLETPRESSURE:200,  #psia
        Parameters.PTProfileSimulation.LIQUIDFLOWRATE:3000,  #stb/d
        Parameters.PTProfileSimulation.FLOWRATETYPE:Constants.FlowRateType.LIQUIDFLOWRATE,
        Parameters.PTProfileSimulation.CALCULATEDVARIABLE:Constants.CalculatedVariable.CUSTOM,
        Parameters.PTProfileSimulation.CUSTOMVARIABLE: {
            Parameters.PTProfileSimulation.CustomVariable.COMPONENT:"FL-1",
            Parameters.PTProfileSimulation.CustomVariable.VARIABLE:"InnerDiameter",
            Parameters.PTProfileSimulation.CustomVariable.MINVALUE:1,
            Parameters.PTProfileSimulation.CustomVariable.MAXVALUE:10,
            }
    }
>>> results = model.tasks.ptprofilesimulation.run(
        producer="Well",
        parameters=parameters,
        profile_variables=profile_variables,
        system_variables=system_variables)
```

For a PT Profile sensitivity study:

```
>>> parameters = {
        Parameters.PTProfileSimulation.INLETPRESSURE:2600,  #psia
        Parameters.PTProfileSimulation.OUTLETPRESSURE:200,  #psia
        Parameters.PTProfileSimulation.CALCULATEDVARIABLE:Constants.CalculatedVariable.FLOWRATE,
        Parameters.PTProfileSimulation.FLOWRATETYPE:Constants.FlowRateType.LIQUIDFLOWRATE,
        Parameters.SingleBranchSimulation.SENSITIVITYVARIABLE: {
            Parameters.SingleBranchSimulation.SensitivityVariable.COMPONENT: "VertComp",
            Parameters.SingleBranchSimulation.SensitivityVariable.VARIABLE:
                Parameters.Completion.RESERVOIRPRESSURE,
            Parameters.SingleBranchSimulation.SensitivityVariable.VALUES: [2000, 3000, 4000]
            }
    }
>>> results = model.tasks.ptprofilesimulation.run(
    producer="Well",
    parameters=parameters,
    system_variables=system_variables,
    profile_variables=profile_variables)
```

If Component is a OneSubsea wet gas compressor and Variable is PressureRatio or Speed, ApplyAll can be set to true such that the sensitivity values are applied to all OneSubsea wet gas compressors in that branch:

```
>>> parameters = {
        Parameters.PTProfileSimulation.INLETPRESSURE:2600,  #psia
        Parameters.PTProfileSimulation.OUTLETPRESSURE:200,  #psia
        Parameters.PTProfileSimulation.CALCULATEDVARIABLE:Constants.CalculatedVariable.FLOWRATE,
        Parameters.PTProfileSimulation.FLOWRATETYPE:Constants.FlowRateType.LIQUIDFLOWRATE,
        Parameters.SingleBranchSimulation.SENSITIVITYVARIABLE: {
            Parameters.SingleBranchSimulation.SensitivityVariable.COMPONENT: "WetGasCompressor",
            Parameters.SingleBranchSimulation.SensitivityVariable.VARIABLE:
                Parameters.Compressor.PRESSURERATIO,
            Parameters.SingleBranchSimulation.SensitivityVariable.VALUES: [1.2, 1.3, 1.4],
            Parameters.SingleBranchSimulation.SensitivityVariable.APPLYALL: True
            }
    }
>>> results = model.tasks.ptprofilesimulation.run(
    producer="Well",
    parameters=parameters,
    system_variables=system_variables,
    profile_variables=profile_variables)
```

With a PT Profile simulation, the list of available components and variables for a producer is found from:

```
>>> model.tasks.ptprofilesimulation.get_sensitivity_variables("Well_1")
```

Nodal analysis simulation is supported in a similar manner.

Get current nodal analysis parameters and inlet conditions for a specified producer and study:

```
>>> general_conds,inlet_conds = model.tasks.nodalanalysis.get_conditions(
        producer="Well", study='Study1')
```

A nodal point can be applied to a nodal analysis simulation when getting the parameters of a nodal analysis study.

For surface nodal point:

```
>>> na_sim, inlet_conds = model.tasks.nodalanalysis.get_conditions(
        "Well 1",
        nodal_point_settings = {
        Parameters.NodalPoint.NODALTYPE: Constants.NodalPointType.SURFACE,
        Parameters.NodalPoint.EQUIPMENT: 'Choke',
        Parameters.NodalPoint.NAME: 'NA1'
    })
```

For downhole nodal point:

```
>>> na_sim, inlet_conds = model.tasks.nodalanalysis.get_conditions(
        "Well 1", "Study 2",
        nodal_point_settings = {
        Parameters.NodalPoint.NODALTYPE: Constants.NodalPointType.DOWNHOLE,
        Parameters.NodalPoint.DEPTH: 1000,
        Parameters.NodalPoint.WELLSTRINGTYPE: Constants.TubingSectionType.TUBING,
        Parameters.NodalPoint.NAME: 'NA2'
    })
```

Update the general and inlet conditions as well as nodal point settings for a nodal analysis simulation:

```
>>> model.tasks.nodalanalysis.set_conditions(
        "Well 1",
        parameters={
            Parameters.NodalAnalysisSimulation.LIMITINFLOW: True,
            Parameters.NodalAnalysisSimulation.LIMITOUTFLOW: False,
            Parameters.NodalAnalysisSimulation.OUTFLOWPOINTS: 40,
            Parameters.NodalAnalysisSimulation.INFLOWPOINTS: 50,
            Parameters.NodalAnalysisSimulation.MAXFLOWRATETYPE:
                Constants.FlowRateType.GASFLOWRATE,
            Parameters.NodalAnalysisSimulation.OUTLETPRESSURE: 3000,
            Parameters.NodalAnalysisSimulation.MAXGASRATE: 300,
            Parameters.NodalAnalysisSimulation.MAXOUTFLOWPRESSURE: 400,
            Parameters.NodalAnalysisSimulation.BRANCHTERMINATOR: "Choke"
        },
        inlet_conditions={
        "Well 1:Comp1":{ Parameters.Boundary.PRESSURE:4000,
                         Parameters.Boundary.TEMPERATURE:200
                       }
        },
        nodal_point_settings={
            Parameters.NodalPoint.NODALTYPE: Constants.NodalPointType.DOWNHOLE,
            Parameters.NodalPoint.DEPTH: 1000,
            Parameters.NodalPoint.WELLSTRINGTYPE: Constants.TubingSectionType.TUBING,
            Parameters.NodalPoint.NAME: 'NA2'
        })
```

Run simulation and get results:

```
>>> sim_results = model.tasks.nodalanalysis.run( "Well 1",
    parameters={
        Parameters.NodalAnalysisSimulation.OUTLETPRESSURE: 3000,
        Parameters.NodalAnalysisSimulation.BRANCHTERMINATOR: "Choke"
    },
    nodal_point_settings = {
        Parameters.NodalPoint.NODALTYPE: Constants.NodalPointType.SURFACE,
        Parameters.NodalPoint.EQUIPMENT: 'Choke',
        Parameters.NodalPoint.NAME: 'NA4'
    },
    profile_variables=OutputVariables.Profile.GAS_FIELD,
    system_variables=OutputVariables.System.GAS_FIELD)
```

Note that system variables that are related to nodal analysis simulation are generated in results by default no matter whether they are provided in system_variables or not.

Additional information speical to nodal analysis simulation such as inflow curves, outflow curves, operating points, operating envelope and fuild base can also be obtained from the results.

Get the information of inflow curve in the first case:

```
>>> result.inflow_curves[0].case
>>> curve = result.inflow_curves[0].curve_data
>>> liquid_curve = curve[SystemVariables.NODAL_POINT_VOLUME_FLOWRATE_LIQUID_STOCKTANK]
>>> mass_curve = curve[SystemVariables.NODAL_POINT_MASS_FLOWRATE_FLUID]
>>> gas_curve = curve[SystemVariables.NODAL_POINT_VOLUME_FLOWRATE_GAS_STOCKTANK]
>>> presure_curve = curve[SystemVariables.NODAL_POINT_PRESSURE]
>>> units = result.inflow_curves[0].units
```

Get the operating envelope for gas phase. Note that operating envelope is generated when there is only one case in simulaiton results.

```
>>> max_drawdown = result.operating_envelope.max_drawdown
>>> max_drawdown[Parameters.NodalOperatingEnvelopePlot.GASFLOWRATE]
>>> max_drawdown[Parameters.NodalOperatingEnvelopePlot.GASFLOWRATEPRESSURE]
>>> res_pres = result.operating_envelope.reservoir_pressure
>>> res_pres[Parameters.NodalOperatingEnvelopePlot.GASFLOWRATE]
>>> res_pres[Parameters.NodalOperatingEnvelopePlot.GASFLOWRATEPRESSURE]
>>> ero_velocity = result.operating_envelope.erosional_velocity
>>> ero_velocity[Parameters.NodalOperatingEnvelopePlot.GASFLOWRATE]
>>> ero_velocity[Parameters.NodalOperatingEnvelopePlot.GASFLOWRATEPRESSURE]
>>> aofp = result.operating_envelope.aofp
>>> aofp[Parameters.NodalOperatingEnvelopePlot.GASFLOWRATE]
>>> aofp[Parameters.NodalOperatingEnvelopePlot.GASFLOWRATEPRESSURE]
>>> inversion_point = result.operating_envelope.inversion_point
>>> inversion_point[Parameters.NodalOperatingEnvelopePlot.GASFLOWRATE]
>>> inversion_point[Parameters.NodalOperatingEnvelopePlot.GASFLOWRATEPRESSURE]
>>> liquid_loading = result.operating_envelope.liquid_loading
>>> liquid_loading[Parameters.NodalOperatingEnvelopePlot.GASFLOWRATE]
>>> liquid_loading[Parameters.NodalOperatingEnvelopePlot.GASFLOWRATEPRESSURE]
```

Get the operating point data for each phase and units in the first case:

```
>>> point = result.operating_points[0].point_data
>>> point[SystemVariables.NODAL_POINT_VOLUME_FLOWRATE_LIQUID_STOCKTANK]
>>> point[SystemVariables.NODAL_POINT_VOLUME_FLOWRATE_GAS_STOCKTANK]
>>> point[SystemVariables.NODAL_POINT_MASS_FLOWRATE_FLUID]
>>> point[SystemVariables.NODAL_POINT_PRESSURE]
>>> units = result.operating_points[0].units
>>> units[SystemVariables.NODAL_POINT_VOLUME_FLOWRATE_LIQUID_STOCKTANK]
>>> units[SystemVariables.NODAL_POINT_VOLUME_FLOWRATE_GAS_STOCKTANK]
>>> units[SystemVariables.NODAL_POINT_MASS_FLOWRATE_FLUID]
>>> units[SystemVariables.NODAL_POINT_PRESSURE]
```

Get the fluid base for nodal analysis operation:

```
>>> fluid_base = result.fluid_base
```

The console messages, summary and state of the simulation after it has finished are properties on the results. This include errors and warnings that you would ordinarily see in the PIPESIM console.

To get the final state of the simulation:

```
>>> results.state
```

A summary of the messages as reported in the PIPESIM UI:

```
>>> results.summary
```

The full console messages as reported by the PIPESIM engine:

```
>>> results.messages
```

Generate PIPESIM engine files in the current folder:

```
>>> model.tasks.networksimulation.generate_engine_files(study="Study 1")
```

To specify the folder for the engine files:

```
>>> model.tasks.ptprofilesimulation.generate_engine_files(
    study="Study 1",
    folder='c:/temp/myproject')
```

The method returns the list of files that has been generated.

Units of Measurement¶

Get the engineering unit and the symbol for a parameter:

```
>>> model.describe(context="Choke", parameter=Parameters.Choke.BEANSIZE).units
>>> model.describe(context="Choke", parameter=Parameters.Choke.BEANSIZE).units_symbol
```

Get the engineering unit conversion for a parameter:

```
>>> model.describe(context="Choke", parameter=Parameters.Choke.BEANSIZE).si_scale
>>> model.describe(context="Choke", parameter=Parameters.Choke.BEANSIZE).si_offset
```

Data Export¶

The model parameters can be exported to a csv file. Note that this does not include the tabulated data such as trajectories at this time:

```
>>> model.export_parameters(filename="C:/Temp/myproject/ExportedParams.csv")
```

Batch Update¶

Operations such as get/set are performed immediately and independently on the model. Each command must finish before the next one can start. There is an overhead on performing each operation through the underlying web API.

The batch update method caches the operations and perform them as a single operation on the web API. This provides improved performance over doing individual operations.

The syntax for using the batch operation is:

```
>>> with model.batch_update():
        model.set_value(
            BlackOilFluid="BK111",
            parameter=Parameters.BlackOilFluid.GOR,
            value=200)
        model.set_value(
            BlackOilFluid="BK111",
            parameter=Parameters.BlackOilFluid.WATERCUT,
            value=5)
        model.set_value(
            BlackOilFluid="BK111",
            parameter=Parameters.BlackOilFluid.GASSPECIFICGRAVITY,
            value=0.7)
        model.set_value(
            BlackOilFluid="BK111",
            parameter=Parameters.BlackOilFluid.WATERSPECIFICGRAVITY,
            value=1.05)
        model.set_value(
            BlackOilFluid="BK111",
            parameter=Parameters.BlackOilFluid.API,
            value=21)
        model.set_value(
            BlackOilFluid="BK111",
            parameter=Parameters.BlackOilFluid.FRACTIONCO2,
            value=0.05)
        model.set_value(
            BlackOilFluid="BK111",
            parameter=Parameters.BlackOilFluid.FRACTIONH2S,
            value=0.001)
        model.set_value(
            BlackOilFluid="BK111",
            parameter=Parameters.BlackOilFluid.FRACTIONN2,
            value=0.1)
```

Utility Functions¶

The following utility functions are included in the PIPESIM Python Toolkit.

Method	Description
convert_tubing_bmd()	Convert bottom measured depth to top measured depth and length.

Convert Bottom Measured Depth¶

The PIPESIM data model uses top measured depth (TMD) and length as the parameters for tubing, casing and liners. Data is often provided, however, as bottom measured depth (BMD). The method converts a list of BMD to a dictionary of TMD and length for setting as parameters.

```
>>> from sixgill.utilities import convert_tubing_bmd
>>> tmd_param = convert_tubing_bmd([2000.0, 2400.0, 2800.0, 3500.0])
>>> model.add(ModelComponents.CASING, "Csg1", context="Well1", parameters=tmd_param[0])
>>> model.add(ModelComponents.TUBING, "Tbl", context="Well1", parameters=tmd_param[1])
```

Exploring the Model Further¶

The Python Toolkit provides complete access to the underlying data model through the function calls. In many areas there are several ways to access the same data.

For instance, to use the surface conditions of a well for the boundaries:

```
>>> model.tasks.networksimulation.use_surface_conditions = True
```

This is also available through:

```
>>> model.set_value(component=ModelComponents.NETWORKSIMULATION,
        Study="Study 1",
        parameter=Parameters.NetworkSimulation.USESURFACEBOUNDARYCONDITIONS,
        value = True)
```

Quick Reference¶

Model Class¶

Properties¶

Property	Description
api	PipesimSession. The web API object connected to PIPESIM
context	AbstractModelContext. The full (root) context of the model
is_open	Boolean. Whether the model is open (True) or not (False)
filename	String. The full path and filename of the model.

Methods¶

Method	Description
about()	Return the details about the model and PIPESIM version
add()	Add a component to the model
batch_update()	Cache the model updates and apply as a single operation
close()	Close the model
connect()	Connect two model components
connections()	List the connections in the model
convert()	Convert a component to a different type
copy()	Copy a model component
delete()	Delete a component from the model
delete_simulation()	Delete the simulation results
describe()	Get the details of a parameter, such as units of measurement
disconnect()	Disconnect two model components
export_well()	Export a well to a PIPESIM model file
export_parameters()	Export the model parameters to a csv file
filter()	Filter the model for a specific context
find()	Find components in the model
get_completion_test_points()	Get the completion test points
get_elevation()	Get the elevation for a latitude / longitude location
get_elevations()	Get elevations for multiple latitude / longitude locations
get_geometry()	Get the flowline geometry
get_geothermal_profile()	Get the geothermal profile
get_pq_curve()	Get the well performance curve
get_trajectory()	Get the well trajectory
get_value()	Get a parameter from a model component
get_values()	Get parameters from model components
import_well()	Import a well from a PIPESIM model
open()	Open a PIPESIM model
open_ui()	Open a PIPESIM model in the PIPESIM UI
read_catalog()	Update the component parameters from the catalog
save()	Save the PIPESIM model
set_all_value()	Set the parameter for all the model components
set_completion_test_points()	Set the completion test points table
set_geometry()	Set the flowline geometry
set_geothermal_profile()	Set the geothermal profile
set_pq_curve()	Set the well performance curve
set_trajectory()	Set the well trajectory
set_value()	Set a parameter on a model component
set_values()	Set a series of parameters on one or more model components
sim_settings()	Get the simulation settings

About Class¶

The properties and methods are accessed by prefixing with *model*.about.

Properties¶

Property	Description
filename	String. The filename of the model.
unit_system	String. The units of measurement system.
ui_unit_system	String. The display units of measurement system in UI.
toolkit_version	String. The release number of the PIPESIM Python Toolkit.
webapi_version	String. The PIPESIM Web API release number

Methods¶

There are no methods on the About() class.

Simulation Settings Class¶

Properties¶

Property	Description
ambient_temperature	Float. The ambient temperature
atmospheric_pressure	Float. The atmospheric pressure.
corrosion_efficiency	Float. The corrosion efficiency
corrosion_model	String. The corrosion model (CorrosionModel static class)
erosional_velocity_constant	Float. The erosional velocity constant.
wind_speed	Float. The wind speed.
soil_type	String. The soil type (SoilType static class).
soil_conductivity	Float. The soil conductivity
metocean_data_location	String. The Metocean data location. (MetoceanDataLocation static class).
max_report_interval_length	Float. The maximum report interval length for pipe segmentation.
print_computation_segment_result	Boolean. Print the computational segment result for pipe segmentation.
max_computation_segment_length	Float. The maximum computational segment length for pipe segmentation.
network_solver_method	String. The network solver method (NetworkSolverMethod static class).
network_solver_tolerance	Float. The network solver tolerance.
network_solver_max_iterations	Integer. The network solver maximum number of iterations.
single_branch_keywords	String. Single branch engine keywords.
network_keywords_top	String. Network engine top keywords.
network_keywords_bottom	String. Network engine bottom keywords.
gis_elevation_interval	Float. GIS elevation interval.
gis_elevation_max_points	Integer. GIS elevation maximum number of points.
gis_elevation_data_source	String. GIS elevation data source (GISDataSource static class).
use_global_flow_correlation	Boolean. Use the global flow correlations.
use_global_heat_transfer	Boolean. Use the global heat transfer settings.

Methods¶

There are no methods on the SimulationSettings() class. The *model*.sim_settings() can return a dictionary, or be treated as a dictionary. See the examples under the Quick Start section.

Fluids Class¶

The properties and methods are accessed by prefixing with *model*.fluids.

Properties¶

Property	Description
fluid_type	The type of fluid used (FluidType static class).

Methods¶

There are no methods on the fluids class.

Compositional Fluid Class¶

The properties and methods are accessed by prefixing with *model*.fluids.compositional

Properties¶

Property	Description
equation_of_state	String. Equation of state (EquationOfState static class).
pvt_package	String. PVT Package (PVTPackage static class).
salinity_model	String. Salinity model (SalinityModel static class).
acf_correlation	String. ACF Correlation (ACFCorrelationMethod static class).
critical_property_correlation	String. Critical Property Correlation (CriticalPropertyCorrelationMethod class).
thermal_coefficient_correlation	String. Thermal Coefficient Correlation (ThermalCoefficientCorrelationMethod class).

Methods¶

Property	Description
add_pseudocomponent()	Add a pseudocomponent.
delete_pseudocomponent()	Delete a pseudocomponent.
select_components()	Select components for the compositional fluid.
unselect_components()	Unselect components for the compositional fluid.

Simulation Tasks Class¶

The properties and methods are accessed by prefixing with *model*.tasks. There is a class for each of the supported tasks.

Classes¶

Class	Description
networksimulation	Methods for running network simulation studies.
networkoptimizersimulation	Methods for running network optimization studies.
ptprofilesimulation	Methods for running PT profile simulation studies.
nodalanalysis	Methods for running nodal analysis simulation studies.
systemanalysis	Methods for running system analysis simulation studies.
wellperformancecurves	Methods for running well performance curves simulation studies.
glddiagnostics	Methods for running gas lift diagnostics simulation studies.

Methods¶

Property	Description
delete_results()	Delete the simulation results.
generate_engine_files()	Generate the engine files for a study
get_calculated_variables()	PT Profile. Get the available calculated variables.
get_conditions()	Get the study boundary conditions.
get_messages()	Get the simulation messages of the simulation.
get_results()	Get the simulation results.
get_sensitivity_variables()	PT Profile. Get the available sensitivity variables.
get_state()	Get the running state of the simulation.
get_use_surface_conditions()	Get whether the model is using surface conditions.
reset_conditions()	Reset the study boundary / inlet conditions.
run()	Run a simulation task and wait for it to finish.
set_conditions()	Set the study boundary conditions or general conditions.
set_use_surface_conditions()	Set whether the model is using surface conditions.
start()	Start a simulation task.

Simulation Result Class¶

Properties¶

Property	Description
system	Dictionary. System results of a simulation run..
node	Dictionary. Node results of a simulation run.
profile	Dictionary. Profile results of a sumulation run.
profile_units	Dictionary. Profile variables and units mapping.
cases	List. A list of sensitivity cases for a single branch simulation run.
messages	List. A list of engine console messages from a simulation run.
summary	Dictionary. A summary of a simulation run status, including errors and warnings.
state	String. The state of a simulation run: RUNNING, COMPLETED, FAILED.

Methods¶

There are no methods on the SimulationResult() class.

Nodal Analysis Result Class¶

A subclass of Simulation Result class containing data for Nodal Plot.

Properties¶

Property	Description
inflow_curves	List. A list of inflow curves, one per each inflow case.
outflow_curves	List. A list of outflow curves, one per each outflow case.
operating_points	List. A list of operation points, one per each sensitivity case.
inflow_cases	List. A list of inflow case names.
outflow_cases	List. A list of outflow case names.
fluid_base	String. The fluid base of nodal simulation results: GAS, LIQUID, or MASS.
operating_envelope	Operating envelope of nodal analysis simulation.

Methods¶

There are no methods on the NodalAnalysisResult() class.

Definitions Sub-Module¶

The definitions sub-module within Sixgill provides static classes for the model operations and parameterization. The enumerations of parameters such as EmulsionViscosityMethod.CONTINUOUSPHASE translating to 'ContinuousPhase' is provided for autocompletion and code editing intellisense.

Static Classes¶

Static Class	Description
Connection	Model component connection ports.
Constants	String enumerations used by the PIPESIM data model.
ModelComponents	Model components.
OutputVariables	Predefined groups of output variables.
Parameters	Parameters for each model component.
ProfileVariables	Output profile variables.
SimulationState	Simulation states.
SystemVariables	Output system variables.
Tasks	Tasks that can be performed on the model.
Units	Available unit systems when opening a model.

Special Keywords¶

The following special string definitions are provided as global variables:

Global Variable	Description
NAN	Mathematical operator "Not a Number" (NaN).
ALL	Match all the items in the context.
NONE	Match none of the items in the context.
TEMPLATE	Match template items in the context.
GLOBAL	The global setting (heat transfer and flow correlations).
DEFAULT	The default setting (heat transfer and flow correlations).

Important Notes¶

Component Conversion¶

The Python Toolkit only supports converting junctions, which can only be converted to a Well, Source or Sink.

Component Copying¶

The copy() method only works on wells at this time.

Table of Contents

- Quick Start
    - Open, Save and Close
    - Query the Model
    - Read and Write Parameters
    - Read and Write Data Tables
    - Add and Delete
    - Copy, Convert, and Rename
    - Connect and Disconnect
    - Manipulate Fluids
    - Import and Export Wells
    - Catalog and GIS
    - Simulation Settings
    - Simulations
    - Units of Measurement
    - Data Export
    - Batch Update
    - Utility Functions
        - Convert Bottom Measured Depth
- Exploring the Model Further
- Quick Reference
    - Model Class
        - Properties
        - Methods
    - About Class
        - Properties
        - Methods
    - Simulation Settings Class
        - Properties
        - Methods
    - Fluids Class
        - Properties
        - Methods
    - Compositional Fluid Class
        - Properties
        - Methods
    - Simulation Tasks Class
        - Classes
        - Methods
    - Simulation Result Class
        - Properties
        - Methods
    - Nodal Analysis Result Class
        - Properties
        - Methods
    - Definitions Sub-Module
        - Static Classes
        - Special Keywords
- Important Notes
    - Component Conversion
    - Component Copying

Previous topic

Changes From Previous Version

Next topic

Working with Excel

Quick search

Go

Navigation

- index
- modules |
- next |
- previous |
- PIPESIM Python Toolkit 2023.1 documentation »
- Quick Start

© Copyright 2023, Schlumberger. Created using Sphinx 5.0.2.